

Stack Overflow Survey Data Cleaning and Normalization

Purpose

This module performs structured cleaning, normalization, and filtering of raw Stack Overflow survey data files from various years.

The result is a consistent dataset across survey formats from 2015 to 2024, suitable for downstream analysis (e.g. salary correlation, language trends, experience mapping).

Key Features

- Year-specific parsing logic for survey CSV files (2015–2024)
 - Developer filtering using occupation and employment status
 - Compensation filtering using IQR with minimum and maximum thresholds
 - Language extraction and normalization
 - Country unification using a manually defined merge map
 - Consistent output format across all survey years
-

Output Structure

All cleaned files are saved in `./CleanedCsvData/` and follow the format:

- `survey_results_public_cleaned_<year>.csv` : Cleaned, filtered data for the given year
 - `all_languages_unique.csv` : Normalized set of all programming languages found
 - `blacklist.csv` : Languages excluded based on non-programmatic or non-general-purpose criteria
 - `all_countries_unique.csv` : List of countries with mapped equivalents for normalization
-

Processing Steps

1. Load Raw Survey CSVs

Raw data from `./RawCsvData/` is processed year by year with schema-specific rules.

2. Filter for Full-Time Developers

Only includes participants with full-time employment and developer-related job roles.

3. Parse and Clean Columns

Salary, experience, country, and language columns are renamed and standardized. Experience is limited to a maximum of 30 years.

4. Outlier Removal (Salary)

Log-based IQR filter is applied to remove unrealistic salary entries. Range constrained to 10,000–300,000 USD.

5. Language Extraction and Normalization

Languages are extracted from semicolon-separated lists and normalized (e.g., "Golang" becomes "Go").

6. Country Mapping

Country names are unified using an explicit merge mapping (e.g., "USA" → "United States").

7. Blacklist Filtering

Languages not meeting inclusion criteria (e.g., HTML, SQL) are removed based on a predefined whitelist.

Inclusion Criteria for Languages

A programming language is considered valid if it:

- Supports control flow structures (`if` , `while` , `function` , etc.)
- Is intended for general-purpose programming
- Is Turing-complete or near-complete in expressiveness

Notes

- Only respondents with valid numeric salaries and experience levels are retained
- Experience levels above 30 years are excluded to avoid outliers
- All parsing is designed to tolerate year-to-year format changes in Stack Overflow surveys

Output Summary

- Cleaned survey data: one CSV per year (2015–2024)
- Language blacklist and whitelist management
- Country name mapping and standardization
- Logging of remaining valid entries (languages and countries) after filtering

Author

Big Data Engineering

FH Technikum Wien, Summer Semester 2025

Manpreet Misson, Timothy Gregorian, Omar Sidi Mammar

Dependencies and Imports

Import core Python libraries for data handling, file operations, parsing, and time management.

```
In [1]: import os
import re
import glob
import numpy as np
import pandas as pd
import time
import ast
import json
from datetime import datetime
from collections import Counter
```

Directory and Filter Configuration

Define directory paths, output files, salary thresholds, and developer-related keyword patterns used for filtering and cleaning survey data.

```
In [2]: RAW_DATA_FOLDER = "./RawCsvData"
CLEANED_FOLDER = "./CleanedCsvData"
LANGUAGE_OUTPUT = os.path.join(CLEANED_FOLDER, "all_languages_unique.csv")
BLACKLIST_FILE = os.path.join(CLEANED_FOLDER, "blacklist.csv")
OUTPUT_FOLDER = "./CleanedCsvData"
MIN_SALARY = 10000
MAX_SALARY = 300000

os.makedirs(CLEANED_FOLDER, exist_ok=True)

DEV_KEYWORDS = [
    "developer", "programmer", "devops", "embedded",
    "database administrator", "systems administrator",
    "machine learning", "data scientist", "statistics", "engineer"
]
DEV_PATTERN = "|".join([re.escape(k) for k in DEV_KEYWORDS])
```

Utility Functions for Data Normalization and Cleaning

These helper functions standardize experience values, filter out salary outliers using IQR, and parse semicolon-separated language lists.

```
In [3]: def parse_years_experience(x):
    if pd.isna(x):
        return np.nan
    x = str(x).strip().lower()
    if re.match(r"(less than|<)\s*\d+", x):
        return 0.0
```

```

if re.match(r"(more than|>|>=|over|more)\s*\d+", x):
    return 31.0
match = re.match(r"(\d+)\s*(?:-|to)\s*(\d+)", x)
if match:
    return (int(match.group(1)) + int(match.group(2))) / 2
match = re.match(r"\d+", x)
if match:
    return float(match.group())
return np.nan

def apply_iqr_filter(df, salary_col="ConvertedCompYearly"):
    df = df[df[salary_col] > 0].copy()
    df["LogSalary"] = np.log(df[salary_col])
    q1 = df["LogSalary"].quantile(0.25)
    q3 = df["LogSalary"].quantile(0.75)
    iqr = q3 - q1
    lower = np.maximum(np.exp(q1 - 1.5 * iqr), MIN_SALARY)
    upper = np.minimum(np.exp(q3 + 1.5 * iqr), MAX_SALARY)
    df = df[(df[salary_col] >= lower) & (df[salary_col] <= upper)].copy()
    return df.drop(columns=["LogSalary"])

def extract_languages(text):
    if pd.isna(text):
        return []
    return [lang.strip() for lang in str(text).split(";") if lang.strip()]

```

Clean Individual Yearly Survey File

Handles year-specific schema variations across raw CSV files, filters for relevant developer roles, standardizes key columns (salary, experience, languages), and exports a cleaned, normalized dataset for each year.

```

In [4]: def clean_file(filepath):
    filename = os.path.basename(filepath)
    year_match = re.search(r"(\d{4})", filename)
    if not year_match:
        print(f"Invalid filename: {filename}")
        return

    year = int(year_match.group(1))
    if year < 2015:
        print(f"Skipping year {year} (before 2015)")
        return

    print(f"\nProcessing {year}...")

    df = pd.read_csv(filepath, low_memory=False)

    if year == 2015:
        df = pd.read_csv(filepath, skiprows=[0], low_memory=False)
        df = df[
            df["Employment Status"].fillna("").eq("Employed full-time") &
            df["Occupation"].fillna("").str.contains(DEV_PATTERN, case=False, re
        ].copy()
        df["MainBranch"] = "I am a developer by profession"
        df = df.rename(columns={
            "Compensation: midpoint": "ConvertedCompYearly",
            "Country": "Country"

```

```

    })
    df["ConvertedCompYearly"] = pd.to_numeric(df["ConvertedCompYearly"], err
    lang_cols = [c for c in df.columns if c.startswith("Current Lang & Tech")
    df["LanguageHaveWorkedWith"] = df[lang_cols].apply(
        lambda row: '; '.join([
            c.replace("Current Lang & Tech: ", "") for c, val in row.items()
            if pd.notna(val) and str(val).strip().lower() not in ["", "0", "
        ]),
        axis=1
    )
    df["YearsCode"] = df["Years IT / Programming Experience"].apply(parse_ye
    df = df[df["YearsCode"] <= 30]

elif year == 2016:
    df = df[
        df["employment_status"].eq("Employed full-time") &
        df["occupation"].fillna("").str.contains(DEV_PATTERN, case=False, re
    ].copy()
    df["MainBranch"] = "I am a developer by profession"
    df = df.rename(columns={
        "salary_midpoint": "ConvertedCompYearly",
        "tech_do": "LanguageHaveWorkedWith",
        "country": "Country",
        "experience_midpoint": "YearsCode"
    })
    df["YearsCode"] = df["YearsCode"].apply(parse_years_experience)
    df = df[df["YearsCode"] <= 30]

elif year == 2017:
    dev_cols = ["DeveloperType", "WebDeveloperType", "MobileDeveloperType",
    df["CombinedDevRoles"] = df[dev_cols].astype(str).agg("; ".join, axis=1)
    df = df[
        df["EmploymentStatus"].fillna("").eq("Employed full-time") &
        df["CombinedDevRoles"].str.contains(DEV_PATTERN, na=False)
    ].copy()
    df["MainBranch"] = "I am a developer by profession"
    df = df.rename(columns={
        "Salary": "ConvertedCompYearly",
        "HaveWorkedLanguage": "LanguageHaveWorkedWith",
        "Country": "Country"
    })
    years_job_col = next((col for col in df.columns if "coded" in col.lower(
    years_prog_col = next((col for col in df.columns if "program" in col.low
    df["YearsCode"] = df[years_job_col].combine_first(df[years_prog_col]).ap
    df = df[df["YearsCode"] <= 30]

elif year == 2018:
    df["CombinedDevRoles"] = df["DevType"].astype(str).str.lower()
    df = df[
        df["Employment"].eq("Employed full-time") &
        df["CombinedDevRoles"].str.contains(DEV_PATTERN, na=False)
    ].copy()
    df["MainBranch"] = "I am a developer by profession"
    df = df.rename(columns={
        "ConvertedSalary": "ConvertedCompYearly",
        "LanguageWorkedWith": "LanguageHaveWorkedWith",
        "Country": "Country",
        "YearsCodingProf": "YearsCode"
    })
    df["YearsCode"] = df["YearsCode"].apply(parse_years_experience)

```

```

df = df[df["YearsCode"] <= 30]

else:
    salary_col = next((c for c in ["ConvertedComp", "ConvertedSalary", "Comp
    lang_col = next((c for c in ["LanguageHaveWorkedWith", "LanguageWorkedWi
    main_col = next((c for c in ["MainBranch", "DevType", "Employment"] if c
    exp_col = next((c for c in ["YearsCode", "YearsCoding", "YearsProgram",
    country_col = "Country" if "Country" in df.columns else None

    if not all([salary_col, lang_col, main_col, exp_col, country_col]):
        print(f"Skipping {year} - missing required columns.")
        return

    df = df[[salary_col, lang_col, main_col, country_col, exp_col]]
    df.columns = ["ConvertedCompYearly", "LanguageHaveWorkedWith", "MainBran
    df["YearsCode"] = df["YearsCode"].apply(parse_years_experience)
    df = df[df["YearsCode"] <= 30]

    if "I am a developer by profession" in df["MainBranch"].unique():
        df = df[df["MainBranch"] == "I am a developer by profession"]
    elif "Employed full-time" in df["MainBranch"].unique():
        df = df[df["MainBranch"] == "Employed full-time"]

df = df[
    df["ConvertedCompYearly"].notna() &
    df["YearsCode"].notna() &
    df["MainBranch"].notna() &
    df["Country"].notna()
].copy()

df["LanguageList"] = df["LanguageHaveWorkedWith"].apply(extract_languages)
df = df[
    df["LanguageList"].notna() &
    df["LanguageList"].apply(lambda x: isinstance(x, list) and len(x) > 0)
]

df["ConvertedCompYearly"] = pd.to_numeric(df["ConvertedCompYearly"], errors=
df = apply_iqr_filter(df)
df["ConvertedCompYearly"] = df["ConvertedCompYearly"].round(1)
df["Year"] = year

df = df[[
    "ConvertedCompYearly",
    "LanguageHaveWorkedWith",
    "MainBranch",
    "Country",
    "YearsCode",
    "LanguageList",
    "Year"
]]
output_file = os.path.join(OUTPUT_FOLDER, f"survey_results_public_cleaned_{y
df.to_csv(output_file, index=False);
print(f"Saved: {output_file} | Rows: {df.shape[0]}");

```

Apply Cleaning Function to All Yearly Survey Files

Iterates through all raw CSV files matching the naming pattern and applies the cleaning function to generate standardized outputs per year.

```
In [5]: for file in sorted(glob.glob(os.path.join(RAW_DATA_FOLDER, "survey_results_public_clean_file(file)
```

Processing 2015...

Saved: ./CleanedCsvData/survey_results_public_cleaned_2015.csv | Rows: 9632

Processing 2016...

Saved: ./CleanedCsvData/survey_results_public_cleaned_2016.csv | Rows: 22756

Processing 2017...

Saved: ./CleanedCsvData/survey_results_public_cleaned_2017.csv | Rows: 9554

Processing 2018...

Saved: ./CleanedCsvData/survey_results_public_cleaned_2018.csv | Rows: 32477

Processing 2019...

Saved: ./CleanedCsvData/survey_results_public_cleaned_2019.csv | Rows: 39498

Processing 2020...

Saved: ./CleanedCsvData/survey_results_public_cleaned_2020.csv | Rows: 23733

Processing 2021...

Saved: ./CleanedCsvData/survey_results_public_cleaned_2021.csv | Rows: 25740

Processing 2022...

Saved: ./CleanedCsvData/survey_results_public_cleaned_2022.csv | Rows: 22782

Processing 2023...

Saved: ./CleanedCsvData/survey_results_public_cleaned_2023.csv | Rows: 31205

Processing 2024...

Saved: ./CleanedCsvData/survey_results_public_cleaned_2024.csv | Rows: 20790

Collect All Languages from Cleaned Files

Scans each cleaned CSV to extract and aggregate all programming languages used, before normalization or filtering.

```
In [6]: all_languages = set()
for file in glob.glob(os.path.join(CLEANED_FOLDER, "survey_results_public_cleaned_2024.csv")):
    df = pd.read_csv(file)
    if "LanguageList" not in df.columns:
        continue
    df["LanguageList"] = df["LanguageList"].apply(ast.literal_eval)
    for langs in df["LanguageList"]:
        all_languages.update(langs)
```

Normalize and Save Language List

Applies consistent naming to variant language labels and exports a unified list of distinct languages.

```
In [7]: normalization_map = {
    "C++11": "C++", "Javascript": "JavaScript", "javascript": "JavaScript",
    "Typescript": "TypeScript", "Golang": "Go", "MATLAB": "Matlab", "Matlab": "Matlab",
    "Perl 6": "Perl", "LISP": "Lisp", "Common Lisp": "Lisp", "Lisp": "Lisp",
```

```

    "Ocaml": "OCaml", "Delphi/Object Pascal": "Delphi",
    "Visual Basic (.Net)": "Visual Basic", "Visual Basic 6": "Visual Basic",
    "Visual Basic 6.0": "Visual Basic", "Visual Basic .NET": "Visual Basic", "VB
}
normalized_languages = set(normalization_map.get(lang, lang) for lang in all_langs)

lang_df = pd.DataFrame(sorted(normalized_languages), columns=["Language"])
lang_df.to_csv(LANGUAGE_OUTPUT, index=False)
print(f"Saved normalized language list to: {LANGUAGE_OUTPUT}")

```

Saved normalized language list to: ./CleanedCsvData/all_languages_unique.csv

Create Language Blacklist

Defines a trusted whitelist and derives a blacklist from languages not included, storing them for later filtering.

```

In [8]: whitelist = {
    "Ada", "APL", "Assembly", "BASIC", "C", "C#", "C++", "COBOL", "Clojure", "Cr
    "D", "Dart", "Delphi", "Elixir", "Elm", "Erlang", "F#", "Fortran", "FreeBASI
    "Groovy", "Hack", "Haskell", "Java", "JavaScript", "Julia", "Kotlin", "Lisp"
    "Matlab", "Nim", "Ocaml", "Objective-C", "Objective-C++", "Pascal", "Perl",
    "PowerShell", "Prolog", "Python", "R", "Raku", "Ruby", "Rust", "Scala", "Sch
    "Smalltalk", "Solidity", "Swift", "TypeScript", "VBA", "Visual Basic", "Zig"
}

blacklist_df = lang_df[~lang_df["Language"].isin(whitelist)]
blacklist_df.to_csv(BLACKLIST_FILE, index=False)
print("Blacklist created.")

blacklist = set(blacklist_df["Language"].str.lower().str.strip())

```

Blacklist created.

Filter and Normalize Languages in Cleaned Files

Applies language normalization and blacklist filtering to each dataset, ensuring only approved languages are retained.

```

In [9]: def clean_languages(text):
    if pd.isna(text):
        return [], ""
    langs = [lang.strip() for lang in str(text).split(";")]
    langs = [normalization_map.get(lang, lang) for lang in langs]
    filtered = [lang for lang in langs if lang.lower() not in blacklist]
    return filtered, ";" + ".join(filtered)

for file in glob.glob(os.path.join(CLEANED_FOLDER, "survey_results_public_cleane
df = pd.read_csv(file)
cleaned = df["LanguageHaveWorkedWith"].apply(clean_languages)
df["LanguageList"] = cleaned.apply(lambda x: x[0])
df["LanguageHaveWorkedWith"] = cleaned.apply(lambda x: x[1])
df = df[df["LanguageList"].apply(lambda x: len(x) > 0)].copy()
df.to_csv(file, index=False)
print(f"Cleaned and saved: {file}")

```



```

Cleaned and saved: ./CleanedCsvData/survey_results_public_cleaned_2023.csv
Cleaned and saved: ./CleanedCsvData/survey_results_public_cleaned_2022.csv
Cleaned and saved: ./CleanedCsvData/survey_results_public_cleaned_2016.csv
Cleaned and saved: ./CleanedCsvData/survey_results_public_cleaned_2017.csv
Cleaned and saved: ./CleanedCsvData/survey_results_public_cleaned_2020.csv
Cleaned and saved: ./CleanedCsvData/survey_results_public_cleaned_2018.csv
Cleaned and saved: ./CleanedCsvData/survey_results_public_cleaned_2021.csv
Cleaned and saved: ./CleanedCsvData/survey_results_public_cleaned_2015.csv
Cleaned and saved: ./CleanedCsvData/survey_results_public_cleaned_2024.csv
Cleaned and saved: ./CleanedCsvData/survey_results_public_cleaned_2019.csv

```

Extract Unique Country Names

Builds a list of all distinct country names from the cleaned datasets and prepares a mapping table for potential normalization.

```

In [10]: COUNTRY_OUTPUT = os.path.join(CLEANED_FOLDER, "all_countries_unique.csv")
all_countries = set()

for file in glob.glob(os.path.join(CLEANED_FOLDER, "survey_results_public_cleaned_*.csv")):
    df = pd.read_csv(file)
    if "Country" not in df.columns:
        continue
    all_countries.update(df["Country"].dropna().unique())

country_df = pd.DataFrame(sorted(all_countries), columns=["OriginalCountry"])
country_df["MappedCountry"] = country_df["OriginalCountry"]
country_df.to_csv(COUNTRY_OUTPUT, index=False)
print(f"Country list saved to: {COUNTRY_OUTPUT}")

```

Country list saved to: ./CleanedCsvData/all_countries_unique.csv

Normalize Country Names via Mapping Table

Applies a predefined mapping to standardize inconsistent or verbose country labels, ensuring consistency across the dataset.

```

In [11]: COUNTRY_MAPPING_FILE = os.path.join(CLEANED_FOLDER, "all_countries_unique.csv")
merge_map = {
    "United States of America": "United States",
    "USA": "United States",
    "United Kingdom of Great Britain and Northern Ireland": "United Kingdom",
    "Republic of Korea": "South Korea",
    "Korea South": "South Korea",
    "Korea, South": "South Korea",
    "Iran, Islamic Republic of...": "Iran",
    "Ireland {Republic}": "Ireland",
    "Hong Kong (S.A.R.)": "Hong Kong",
    "Bosnia-Herzegovina": "Bosnia and Herzegovina",
    "Bosnia Herzegovina": "Bosnia and Herzegovina",
    "Viet Nam": "Vietnam",
    "Moldavia": "Moldova",
    "Republic of Moldova": "Moldova",
    "Libyan Arab Jamahiriya": "Libya",
    "Ivory Coast": "Côte d'Ivoire",
    "Myanmar, {Burma}": "Myanmar",
    "Syria": "Syrian Arab Republic",
}

```

```

    "Democratic People's Republic of Korea": "North Korea",
    "Lao People's Democratic Republic": "Laos",
    "Slovak Republic": "Slovakia",
    "The former Yugoslav Republic of Macedonia": "North Macedonia",
    "Republic of North Macedonia": "North Macedonia",
    "Trinidad & Tobago": "Trinidad and Tobago",
    "Russian Federation": "Russia",
    "Venezuela, Bolivarian Republic of...": "Venezuela"
}

country_df = pd.read_csv(COUNTRY_MAPPING_FILE)
country_df["MappedCountry"] = country_df["MappedCountry"].replace(merge_map)
country_df.to_csv(COUNTRY_MAPPING_FILE, index=False)
print(f"Updated MappedCountry saved to: {COUNTRY_MAPPING_FILE}")

```

Updated MappedCountry saved to: ./CleanedCsvData/all_countries_unique.csv

Apply Normalized Country Mapping to Cleaned CSVs

Replaces original country entries with standardized names across all cleaned survey files using the constructed alias dictionary.

```

In [12]: country_aliases = dict(zip(country_df["OriginalCountry"], country_df["MappedCountry"]))

for file in glob.glob(os.path.join(CLEANED_FOLDER, "survey_results_public_cleaned_*.csv")):
    df = pd.read_csv(file)
    if "Country" not in df.columns:
        continue
    df["Country"] = df["Country"].map(lambda x: country_aliases.get(x, x) if pd.notna(x) else None)
    df.to_csv(file, index=False)
    print(f"Country mapping applied: {file}")

```

Country mapping applied: ./CleanedCsvData/survey_results_public_cleaned_2023.csv
 Country mapping applied: ./CleanedCsvData/survey_results_public_cleaned_2022.csv
 Country mapping applied: ./CleanedCsvData/survey_results_public_cleaned_2016.csv
 Country mapping applied: ./CleanedCsvData/survey_results_public_cleaned_2017.csv
 Country mapping applied: ./CleanedCsvData/survey_results_public_cleaned_2020.csv
 Country mapping applied: ./CleanedCsvData/survey_results_public_cleaned_2018.csv
 Country mapping applied: ./CleanedCsvData/survey_results_public_cleaned_2021.csv
 Country mapping applied: ./CleanedCsvData/survey_results_public_cleaned_2015.csv
 Country mapping applied: ./CleanedCsvData/survey_results_public_cleaned_2024.csv
 Country mapping applied: ./CleanedCsvData/survey_results_public_cleaned_2019.csv

Summary Output: Remaining Countries in Cleaned Dataset

This block counts and displays how many records exist for each country after all cleaning and country mapping steps have been applied.

```

In [13]: country_counter = Counter()
for file in glob.glob(os.path.join(CLEANED_FOLDER, "survey_results_public_cleaned_*.csv")):
    df = pd.read_csv(file)
    if "Country" in df.columns:
        country_counter.update(df["Country"].dropna())

print("\nRemaining countries in the cleaned dataset:")
for country, count in sorted(country_counter.items()):
    print(f"{country}: {count}")

```

```
print(f"- {country}: {count}x")  
print(f"\nTotal: {len(country_counter)} unique countries after filtering.")
```

Remaining countries in the cleaned dataset:

- Afghanistan: 39x
- Albania: 107x
- Algeria: 59x
- Andorra: 19x
- Angola: 8x
- Antigua & Deps: 1x
- Antigua and Barbuda: 1x
- Argentina: 1397x
- Armenia: 156x
- Australia: 5924x
- Austria: 2520x
- Azerbaijan: 69x
- Bahamas: 6x
- Bahrain: 29x
- Bangladesh: 728x
- Barbados: 11x
- Belarus: 413x
- Belgium: 2048x
- Belize: 4x
- Benin: 8x
- Bermuda: 1x
- Bhutan: 5x
- Bolivia: 80x
- Bosnia and Herzegovina: 222x
- Botswana: 9x
- Brazil: 5734x
- Brunei Darussalam: 1x
- Bulgaria: 1175x
- Burundi: 1x
- Cambodia: 27x
- Cameroon: 23x
- Canada: 11185x
- Cape Verde: 3x
- Cayman Islands: 1x
- Central African Republic: 1x
- Chile: 358x
- China: 1110x
- Colombia: 529x
- Congo, Republic of the...: 5x
- Costa Rica: 218x
- Croatia: 783x
- Cuba: 36x
- Cyprus: 186x
- Czech Republic: 1765x
- Côte d'Ivoire: 19x
- Democratic Republic of the Congo: 6x
- Denmark: 1706x
- Djibouti: 1x
- Dominica: 1x
- Dominican Republic: 264x
- East Timor: 1x
- Ecuador: 170x
- Egypt: 433x
- El Salvador: 97x
- Estonia: 559x
- Ethiopia: 83x
- Fiji: 9x
- Finland: 1774x
- France: 9148x

- Gabon: 1x
- Gambia: 1x
- Georgia: 253x
- Germany: 18814x
- Ghana: 82x
- Greece: 1512x
- Grenada: 1x
- Guam: 1x
- Guatemala: 145x
- Guinea: 1x
- Guyana: 5x
- Haiti: 4x
- Honduras: 55x
- Hong Kong: 371x
- Hungary: 870x
- I prefer not to say: 3x
- Iceland: 86x
- India: 8818x
- Indonesia: 265x
- Iran: 704x
- Iraq: 25x
- Ireland: 1683x
- Isle of Man: 27x
- Israel: 2331x
- Italy: 4838x
- Jamaica: 31x
- Japan: 694x
- Jordan: 140x
- Kazakhstan: 115x
- Kenya: 282x
- Kosovo: 36x
- Kuwait: 25x
- Kyrgyzstan: 51x
- Laos: 6x
- Latvia: 437x
- Lebanon: 176x
- Lesotho: 5x
- Libya: 5x
- Liechtenstein: 8x
- Lithuania: 766x
- Luxembourg: 172x
- Macedonia: 27x
- Madagascar: 12x
- Malawi: 5x
- Malaysia: 540x
- Maldives: 28x
- Mali: 3x
- Malta: 213x
- Martinique (French): 1x
- Mauritania: 4x
- Mauritius: 53x
- Mexico: 1720x
- Moldova: 113x
- Monaco: 7x
- Mongolia: 15x
- Montenegro: 49x
- Morocco: 208x
- Mozambique: 11x
- Myanmar: 17x
- Namibia: 11x

- Nepal: 214x
- Netherlands: 5782x
- New Caledonia (French): 2x
- New Zealand: 1786x
- Nicaragua: 61x
- Niger: 2x
- Nigeria: 348x
- Nomadic: 52x
- North Korea: 2x
- North Macedonia: 95x
- Norway: 1147x
- Oman: 18x
- Other (please specify): 4x
- Other Country (Not Listed Above): 59x
- Pakistan: 1037x
- Palau: 1x
- Palestine: 20x
- Panama: 85x
- Paraguay: 97x
- Peru: 258x
- Philippines: 491x
- Poland: 6186x
- Portugal: 1968x
- Puerto Rico: 4x
- Qatar: 38x
- Reunion (French): 1x
- Romania: 2138x
- Russia: 3907x
- Rwanda: 15x
- Saint Kitts and Nevis: 2x
- Saint Vincent and the Grenadines: 2x
- Samoa: 1x
- San Marino: 1x
- Saudi Arabia: 184x
- Senegal: 11x
- Serbia: 779x
- Seychelles: 1x
- Sierra Leone: 2x
- Singapore: 750x
- Slovakia: 622x
- Slovenia: 759x
- Somalia: 10x
- South Africa: 1693x
- South Korea: 364x
- Spain: 6231x
- Sri Lanka: 424x
- Sudan: 17x
- Suriname: 7x
- Swaziland: 9x
- Sweden: 3808x
- Switzerland: 3254x
- Syrian Arab Republic: 17x
- Taiwan: 387x
- Tajikistan: 3x
- Thailand: 342x
- Timor-Leste: 1x
- Togo: 8x
- Trinidad and Tobago: 42x
- Tunisia: 95x
- Turkey: 2078x

- Turkmenistan: 17x
- U.S. Minor Outlying Islands: 1x
- Uganda: 43x
- Ukraine: 2758x
- United Arab Emirates: 425x
- United Kingdom: 20053x
- United Republic of Tanzania: 25x
- United States: 67170x
- Uruguay: 373x
- Uzbekistan: 45x
- Venezuela: 109x
- Vietnam: 226x
- Virgin Islands (USA): 1x
- Yemen: 11x
- Zambia: 23x
- Zimbabwe: 34x

Total: 195 unique countries after filtering.

Output Summary – Remaining Programming Languages

This section tallies how often each programming language appears in the cleaned dataset across all years after filtering and normalization.

```
In [14]: counter = Counter()
for file in glob.glob(os.path.join(CLEANED_FOLDER, "survey_results_public_cleaned")):
    df = pd.read_csv(file)
    df["LanguageList"] = df["LanguageList"].apply(ast.literal_eval)
    for langs in df["LanguageList"]:
        counter.update(langs)

print("\nRemaining programming languages in the cleaned dataset:")
for lang, count in sorted(counter.items()):
    print(f"- {lang}: {count}x")
print(f"\nTotal: {len(counter)} unique languages after filtering.")
```

Remaining programming languages in the cleaned dataset:

- APL: 144x
- Ada: 225x
- Assembly: 7036x
- C: 32379x
- C#: 78626x
- C++: 41761x
- Clojure: 3533x
- Crystal: 397x
- Dart: 6494x
- Delphi: 2235x
- Elixir: 3698x
- Erlang: 1828x
- F#: 2533x
- Fortran: 371x
- Go: 24321x
- Groovy: 6609x
- Hack: 96x
- Haskell: 3034x
- Java: 81991x
- JavaScript: 163783x
- Julia: 1036x
- Kotlin: 15850x
- Lisp: 1061x
- Lua: 4726x
- Matlab: 4711x
- Nim: 108x
- OCaml: 471x
- Objective-C: 11118x
- PHP: 52364x
- Perl: 5469x
- PowerShell: 13908x
- Prolog: 245x
- Python: 90997x
- R: 8705x
- Raku: 27x
- Ruby: 22192x
- Rust: 13887x
- Scala: 9540x
- Smalltalk: 66x
- Solidity: 825x
- Swift: 14111x
- TypeScript: 71273x
- VBA: 7811x
- Visual Basic: 7994x
- Zig: 372x

Total: 45 unique languages after filtering.